

Aspect-Based Sentiment Analysis

(Task 2 Report)

Amartya Singh (2022062), Anish(2022075), Adarsh Jha (2022024)

March 16, 2025

Contents

1	Dataset and Preprocessing	2
1.1	Original Data	2
1.2	Preprocessing Steps	2
1.3	Example of <code>train_task_2.json</code>	2
2	Model Architectures	3
2.1	Custom RNN + GRU + LSTM + Interactive Attention	3
2.2	Fine-Tuned Transformer Models	4
2.2.1	BERT	4
2.2.2	BART	4
2.2.3	RoBERTa	5
3	Training Details	5
4	Results and Analysis	5
4.1	Custom Model: Accuracy and Loss	5
4.2	BERT Fine-Tuning	6
4.3	BART Fine-Tuning	6
4.4	RoBERTa Fine-Tuning	7
4.5	Model Comparison on Test Set	7
5	Conclusion	8

1 Dataset and Preprocessing

1.1 Original Data

- We are given `train.json` and `val.json`, where each entry contains:
 - A sentence (text).
 - A list of aspect terms (e.g., “food”, “service”, etc.) with their corresponding polarity (*positive*, *negative*, or *neutral*).
 - Character indices of each aspect term within the sentence.

1.2 Preprocessing Steps

- **Goal:** Treat each aspect term as a *separate training instance*.
- **Implementation:**
 1. Tokenize the sentence into words or subwords.
 2. For each aspect term in the sentence, create a new example containing:
 - `tokens`: The entire tokenized sentence.
 - `polarity`: The sentiment of this specific aspect term.
 - `aspect_term`: A list or string containing the exact aspect term.
 - `index`: The position of the aspect term in the tokenized sentence.
- The final files are `train_task_2.json` and `val_task_2.json`.

1.3 Example of `train_task_2.json`

```
{
  "tokens": ["The", "food", "is", "uniformly", "exceptional", "..."],
  "polarity": "positive",
  "aspect_term": ["food"],
  "index": 1
}
{
  "tokens": ["The", "food", "is", "uniformly", "exceptional", "..."],
  "polarity": "positive",
  "aspect_term": ["kitchen"],
  "index": 9
}
...
```

Each aspect term from the original sentence becomes a separate entry. This way, we directly learn the sentiment for each term.

2 Model Architectures

2.1 Custom RNN + GRU + LSTM + Interactive Attention

Purpose and Motivation:

- Exploit the strengths of multiple recurrent variants (RNN, GRU, LSTM).
- Use an *interactive attention* mechanism to highlight tokens most relevant to the aspect term.

Detailed Explanation:

1. Embedding Layer:

- We use pretrained embeddings (e.g., GloVe) to initialize the word vectors.
- Each token is mapped to a d -dimensional vector (e.g., $d = 300$).

2. Stacked Recurrent Layers:

- **RNN (vanilla):** Encodes basic sequential context.
- **GRU:** Adds gating mechanisms to handle longer sequences without vanishing gradients.
- **LSTM:** Further enhances long-term memory via input, output, and forget gates.

Typically, the hidden state output from one layer is fed into the next, forming a stacked architecture:

$$\text{Embeddings} \rightarrow \text{RNN} \rightarrow \text{GRU} \rightarrow \text{LSTM}.$$

3. Interactive Attention:

- We produce a hidden state $\mathbf{h}_t$ at each time step t of the LSTM.
- The aspect term embedding (or an average of embeddings if the aspect term has multiple tokens) is used to compute attention scores:

$$\alpha_t = \frac{\exp(\text{score}(\mathbf{h}_t, \mathbf{a}))}{\sum_k \exp(\text{score}(\mathbf{h}_k, \mathbf{a}))},$$

where $\mathbf{a}$ is the aspect embedding, and $\text{score}(\cdot)$ can be a dot product or a learned function.

- The weighted sum of hidden states:

$$\mathbf{c} = \sum_t \alpha_t \mathbf{h}_t,$$

is a *context vector* focusing on tokens relevant to the aspect term.

4. Classification Layer:

- We feed the context vector $\mathbf{c}$ into a fully connected layer with softmax to classify {positive, negative, neutral}.
- Cross-entropy loss is used to train the model.

2.2 Fine-Tuned Transformer Models

2.2.1 BERT

- **Architecture:** 12 transformer encoder blocks, each with multi-head self-attention.
- **Input Representation:**
 - We prepend [CLS] at the start of the sentence and append [SEP] at the end.
 - The sentence might look like: [CLS] The food is exceptional ... [SEP].
- **Output:**
 - The final hidden state of [CLS] is passed to a linear layer for classification.
 - We fine-tune all layers of BERT plus the final classifier for sentiment polarity.

2.2.2 BART

- **Encoder-Decoder:** BART is typically used for sequence-to-sequence tasks, but can also be adapted for classification.
- **Process:**
 - The input sentence is encoded via BART's encoder.
 - We use a classification head on top of the encoder's output (often the first token's hidden state or a pooled representation).
 - All parameters are fine-tuned jointly for the aspect polarity classification objective.

2.2.3 RoBERTa

- **Improved BERT Training:** RoBERTa modifies BERT's pretraining with dynamic masking, longer training, and larger mini-batches.
- **Classification Head:**
 - Similar to BERT, we use the [CLS] hidden state (referred to as <s> in RoBERTa).
 - A linear layer + softmax outputs the polarity.

3 Training Details

- **Optimizer:** Adam (default parameters).
- **Loss Function:** Cross-entropy.
- **Learning Rate:**
 - Custom model: $1e-3$
 - Transformers: $2e-5$
- **Epochs / Steps:**
 - Custom model: 3–5 epochs.
 - Transformers: 400–500 steps, depending on dataset size.
- **Batch Size:** 16

4 Results and Analysis

4.1 Custom Model: Accuracy and Loss

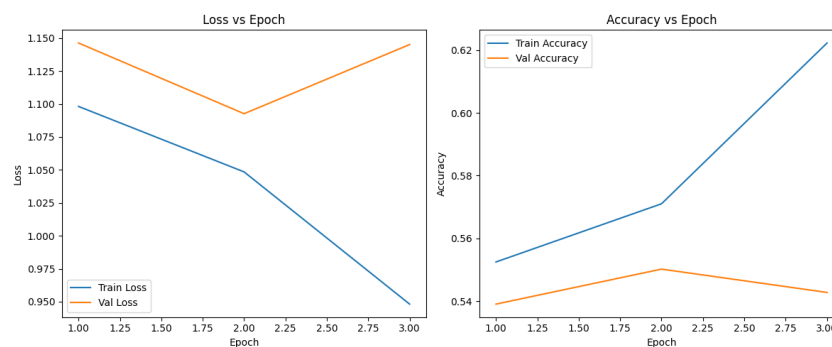


Figure 1: Custom Model (RNN+GRU+LSTM+Attention) Accuracy and Loss.

Observations:

- The *training loss* steadily decreases, showing that the model is learning the task.
- The *validation loss* decreases initially but then slightly increases, hinting at mild overfitting.
- Accuracy curves follow a similar trend, with training accuracy consistently higher than validation accuracy by a small margin.

4.2 BERT Fine-Tuning

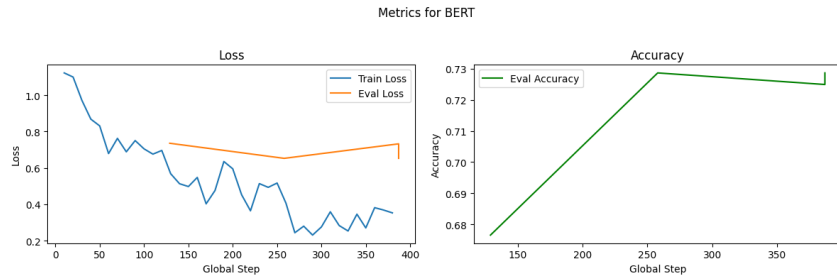


Figure 2: BERT Training/Validation Accuracy and Loss.

Observations:

- BERT's *training loss* decreases steadily over steps.
- *Validation accuracy* shows a clear upward trend, from around 0.68 to about 0.71–0.72.
- Slight fluctuations in validation loss are normal for transformer fine-tuning but overall it trends downward.

4.3 BART Fine-Tuning

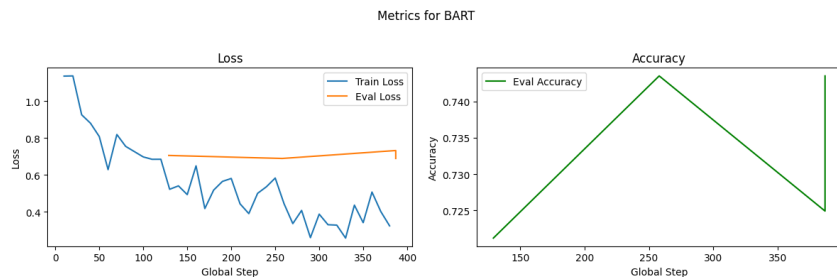


Figure 3: BART Training/Validation Accuracy and Loss.

Observations:

- Training loss starts higher (around 1.0+) and decreases steadily.

- Validation accuracy peaks near 0.74, then slightly dips, which may indicate the onset of overfitting.

4.4 RoBERTa Fine-Tuning

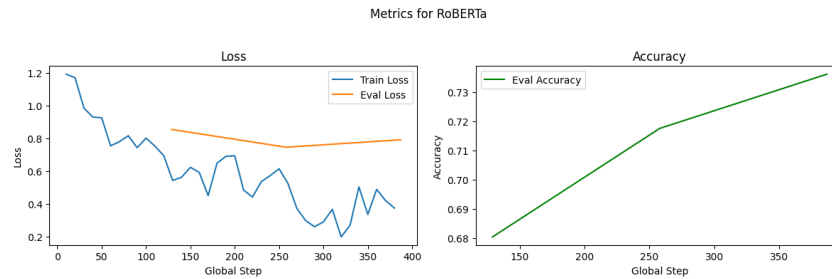


Figure 4: RoBERTa Training/Validation Accuracy and Loss.

Observations:

- Training loss decreases from around 1.2 down to about 0.3–0.4.
- Validation accuracy climbs to around 0.73–0.75, showing strong performance.

4.5 Model Comparison on Test Set

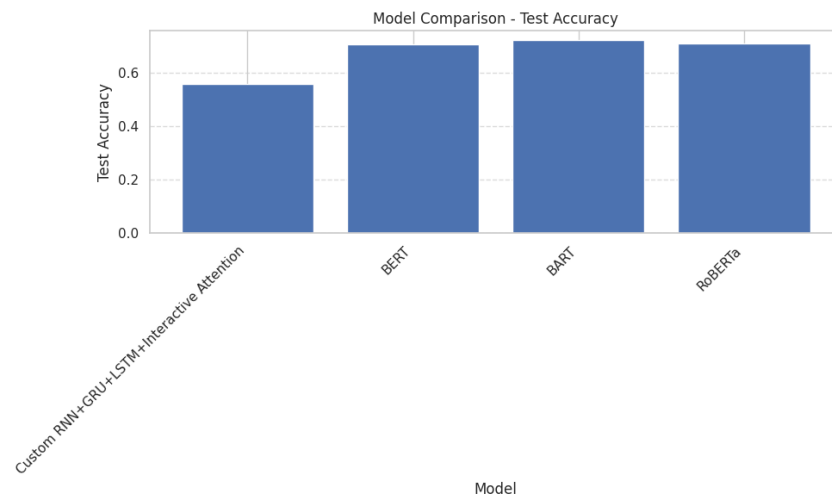


Figure 5: Comparison of Test Accuracies for All Models.

Detailed Final Results (with inference time per sample):

- **BART**: Accuracy = 0.7204, Time = 3.28s
- **RoBERTa**: Accuracy = 0.7105, Time = 2.62s

- **BERT**: Accuracy = 0.7072, Time = 2.81s
- **Custom RNN+GRU+LSTM+Interactive Attention**: Accuracy = 0.5576, Time = 1.19s

Analysis of Final Comparison:

- **BART** yields the highest accuracy (0.7204).
- **RoBERTa** is close behind (0.7105) with a slightly faster inference time.
- **BERT** sits at 0.7072 accuracy, also reasonably fast.
- The **Custom Model** achieves 0.5576, significantly lower, but with the fastest inference time among the four.

5 Conclusion

- The **Custom RNN+GRU+LSTM+Attention** model learns aspect-level sentiment but underperforms compared to large-scale transformers.
- Among **BERT**, **BART**, and **RoBERTa**, **BART** achieved the best final test accuracy, followed closely by RoBERTa and BERT.
- Inference speed is generally slower for BART but yields slightly higher accuracy. RoBERTa and BERT are close in accuracy and speed.
- Further tuning (e.g., domain-specific pretraining, advanced attention mechanisms) may improve results for all models.

The fine tuned models couldn't get saved due to time constraints, hence they are saved in the google drive folder which can be accessed by the following link :

<https://drive.google.com/drive/folders/1-xgx62COLOV0pFp3gJyNXcoT96O5xPQ?usp=sharing>